INSTITUTO FEDERAL DE EDUCAÇÃO
DISCIPLINA: Padrões de Projeto em Aplicações Web
PROFESSOR: Éder Pereira
ATIVIDADE PRÁTICA: Padrão Criacional Factory Method

Contextualização do Problema

Sistemas corporativos modernos necessitam comunicar-se constantemente com seus clientes e outros subsistemas por meio de múltiplos canais (como SMS, E-mail e WhatsApp). Em uma arquitetura bancária ou de notificações, acoplar o código do cliente diretamente às instâncias concretas de cada canal de envio gera alta dependência e dificulta a manutenção e a inclusão de novos canais.

Para solucionar este problema de acoplamento rígido, utilizaremos o padrão de projeto criacional **Factory Method**. A ideia central é encapsular a lógica de criação dos objetos em uma classe intermediária (`EmissorCreator`), permitindo que o sistema solicite a criação de um emissor sem precisar conhecer os detalhes de instanciação das classes concretas (`EmissorSMS`, `EmissorEmail`, `EmissorWhatsApp`).

Roteiro de Implementação

1. **Criação do Projeto:**
   - Crie um projeto Java no Eclipse chamado `FactoryMethod`.
2. **Definição da Interface `Emissor`:**
   - Crie o arquivo `Emissor.java` com a assinatura do método de envio.

```java
public interface Emissor {
    void envia(String mensagem);
}
```

3. **Definição das Classes Concretas:**
   - Crie as classes `EmissorSMS`, `EmissorEmail` e `EmissorWhatsApp` que devem implementar a interface `Emissor`.

```java
public class EmissorSMS implements Emissor {
    // TODO: Implementar o metodo envia para SMS
}

public class EmissorEmail implements Emissor {
    // TODO: Implementar o metodo envia para Email
}

public class EmissorWhatsApp implements Emissor {
    // TODO: Implementar o metodo envia para WhatsApp
}
```

4. **Criação da Classe Fábrica (`EmissorCreator`):**
   - Implemente a classe responsável por instanciar os objetos corretos com base em constantes inteiras.

```java
public class EmissorCreator {
    public static final int SMS = 0;
    public static final int EMAIL = 1;
    public static final int WHATSAPP = 2;
```

```
5
6       public Emissor create(int tipoDeEmissor) {
7           // TODO: Implementar a logica de decisao e criacao das instancias
8           return null;
9       }
10  }
```

5. **Testes do Sistema (`TestaEmissores`):**

   - Desenvolva a classe de testes utilizando a fábrica para criar os emissores sem acoplamento direto com as classes concretas.

```
1   public class TestaEmissores {
2       public static void main(String[] args) {
3           // TODO: Instanciar o EmissorCreator
4           // TODO: Solicitar a criacao dos emissores (SMS, EMAIL, WHATSAPP)
                via fabrica
5           // TODO: Executar o metodo envia() para cada emissor criado
6       }
7   }
```